

Machine Learning - HW #2

Ensemble Learning for Imbalanced Classification:

COVID-19 Severity Prediction Dataset

Course	Machine Learning
Homework	Ensemble Learning for Imbalanced Classification on COVID-19 Severity Prediction Dataset
Students	Mana Peiravian – 40464250 Sina Sabooki – 40435072

This report follows the required pipeline and experiments: train-only preprocessing, stratified 70/30 splits, 10 independent runs using seeds 0-9, minority-class metrics for label +1, AUC-ROC, and G-mean. Source files are included separately in the code folder.

1. Dataset and Preprocessing

The dataset contains 1,585 patients, 40 input features, and a binary Label column. The positive class (+1, severe) has 100 cases and the negative class (-1, non-severe) has 1,485 cases, giving an imbalance ratio of about 15:1. This means accuracy alone is misleading: predicting every patient as non-severe would achieve about 93.7% accuracy while giving zero recall and zero G-mean for severe cases.

Table 1. Dataset overview and split check

Item	Value
Samples	1585
Features	40
Severe (+1)	100
Non-severe (-1)	1485
Imbalance ratio	14.85:1
Train split at seed 42	1109 samples: 70 severe, 1039 non-severe
Test split at seed 42	476 samples: 30 severe, 446 non-severe

Missing values were imputed using training-set statistics only. Continuous features age and PO2 were imputed with the median because medians are robust to skew and outliers. Binary/categorical symptom and comorbidity indicators were imputed with the mode because their valid values are categorical indicator states. This avoids leakage because imputation parameters are estimated only from each training fold and then applied to its test fold.

Table 2. Missing-value rate by column

Column	Missing (%)
dizziness	82.08
paresis	82.08
plexus	82.08
inflammation / skin lesions	82.08
Chest pain	82.08
headache	82.08
nausea	64.54
Stomachache	64.54
loss of appetite	64.54
vomiting	64.54
diarrhea	64.54
Convulsions	1.70
loss of taste	1.70
loss of smell	0.00
Fever	0.00
contact with patient	0.00
gender	0.00
age	0.00
Inability to wake	0.00
Shortness of breath	0.00
Muscular pain	0.00
Cough	0.00
smoking	0.00
History of drug abuse (opium)	0.00
Intubation	0.00
PO2	0.00
cancer	0.00
Chronic liver disease	0.00
Diabetes	0.00
Chronic blood diseases	0.00
HIV/AIDS	0.00
Immune Deficiency (Acquired or Congenital)	0.00
Pregnancy	0.00
Heart disease	0.00
Chronic kidney disease	0.00
Asthma	0.00
Other chronic lung diseases	0.00
Chronic neurological disorders	0.00
Other chronic diseases	0.00
History of hypertension	0.00
Label	0.00

Feature selection used Kendall rank correlation on each training split. For every pair of features with $|\tau| > 0.5$, one feature was removed, keeping the one with the higher absolute Kendall correlation with the label. At the seed-42 split, 39 of 40 features remained; the dropped feature was: vomiting. StandardScaler was fitted only to the selected continuous columns (age, PO2) on the training data and then used to transform the test data.

2. Task 1 - Hellinger Distance Decision Tree (HDDT)

The HDDT implementation is recursive and does not use scikit-learn decision-tree internals. At every node, it evaluates all midpoint thresholds between sorted unique feature values and

selects the split maximizing the Hellinger score $H_f = \left(\sqrt{\frac{p_L}{P}} - \sqrt{\frac{n_L}{N}} \right)^2 + \left(\sqrt{\frac{p_R}{P}} - \sqrt{\frac{n_R}{N}} \right)^2$. Leaves predict the majority class at the leaf, and predict_proba returns the fraction of positive samples at the leaf.

Table 3. HDDT evaluation with max_depth=None

Metric	Mean (10 runs)	Std Dev
Accuracy	0.9109	0.0138
Precision (+1)	0.1562	0.0945
Recall (+1)	0.0833	0.0503
F1 (+1)	0.1023	0.0564
AUC-ROC	0.5846	0.0186
G-mean	0.2710	0.0868

The fully grown HDDT achieves lower overall accuracy than the trivial majority predictor but detects some severe cases. Its minority recall is still modest, which is expected with only about 70 positive training examples per split.

Table 4. HDDT pruning experiment: mean $\pm$ std over 10 runs

max_depth	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	AUC-ROC	G-mean
None	0.9109 $\pm$ 0.0138	0.1562 $\pm$ 0.0945	0.0833 $\pm$ 0.0503	0.1023 $\pm$ 0.0564	0.5846 $\pm$ 0.0186	0.2710 $\pm$ 0.0868
2	0.9370 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.5439 $\pm$ 0.0109	0.0000 $\pm$ 0.0000
3	0.9366 $\pm$ 0.0013	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.5560 $\pm$ 0.0161	0.0000 $\pm$ 0.0000
4	0.9353 $\pm$ 0.0041	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.5742 $\pm$ 0.0299	0.0000 $\pm$ 0.0000
5	0.9340 $\pm$ 0.0039	0.0250 $\pm$ 0.0791	0.0033 $\pm$ 0.0105	0.0059 $\pm$ 0.0186	0.5818 $\pm$ 0.0299	0.0182 $\pm$ 0.0575

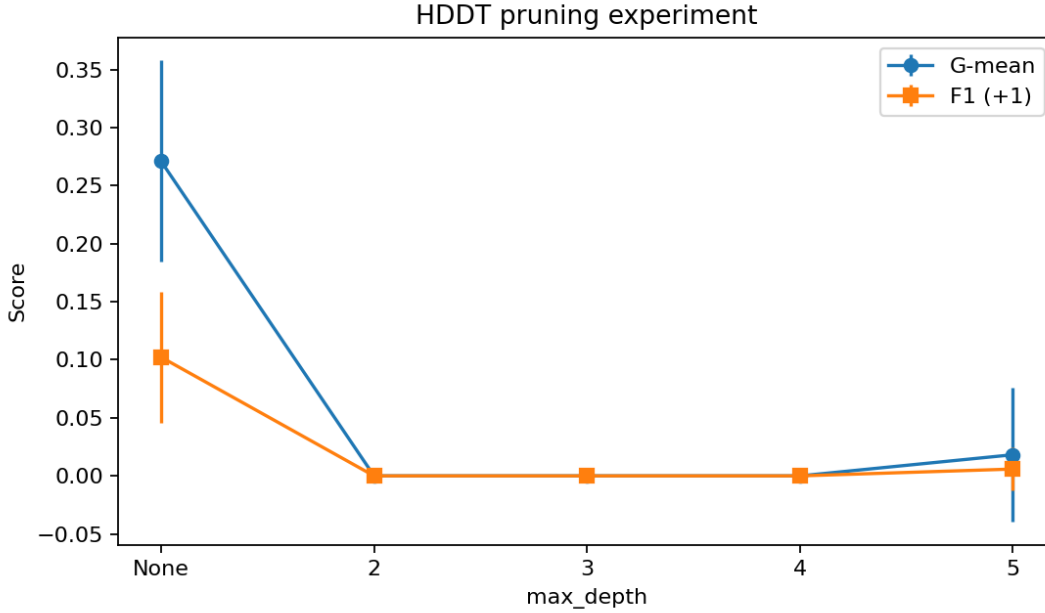


Figure 1. HDDT pruning experiment: G-mean and F1 versus max_depth.

In these runs, shallow trees with depths 2-4 mostly predicted the majority class, producing near-zero F1 and G-mean despite high accuracy. The unrestricted tree gave the best F1 and G-mean, while depth 5 began to recover a very small number of positives. For this dataset and implementation, the best bias-variance tradeoff by minority detection was therefore the unrestricted HDDT, although its variance and precision remain limited.

3. Task 2 - Bagging with Undersampling

Each ensemble member bootstraps the minority class with replacement, randomly undersamples the majority class without replacement to the same size, trains a base learner on the resulting balanced subset, and combines predictions by majority vote. Predicted probabilities are averaged over all ensemble members.

Table 5. Bagging + HDDT (max_depth=3): effect of ensemble size T

T	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	AUC-ROC	G-mean
11	0.2674 ± 0.1353	0.0728 ± 0.0075	0.8933 ± 0.1386	0.1342 ± 0.0125	0.6513 ± 0.0588	0.4198 ± 0.1005
31	0.2357 ± 0.0458	0.0732 ± 0.0051	0.9500 ± 0.0283	0.1358 ± 0.0090	0.6694 ± 0.0545	0.4192 ± 0.0562
51	0.2477 ± 0.0877	0.0743 ± 0.0081	0.9433 ± 0.0473	0.1376 ± 0.0134	0.6648 ± 0.0460	0.4247 ± 0.0868
101	0.2897 ± 0.1169	0.0766 ± 0.0092	0.9133 ± 0.0958	0.1410 ± 0.0150	0.6704 ± 0.0458	0.4561 ± 0.1020

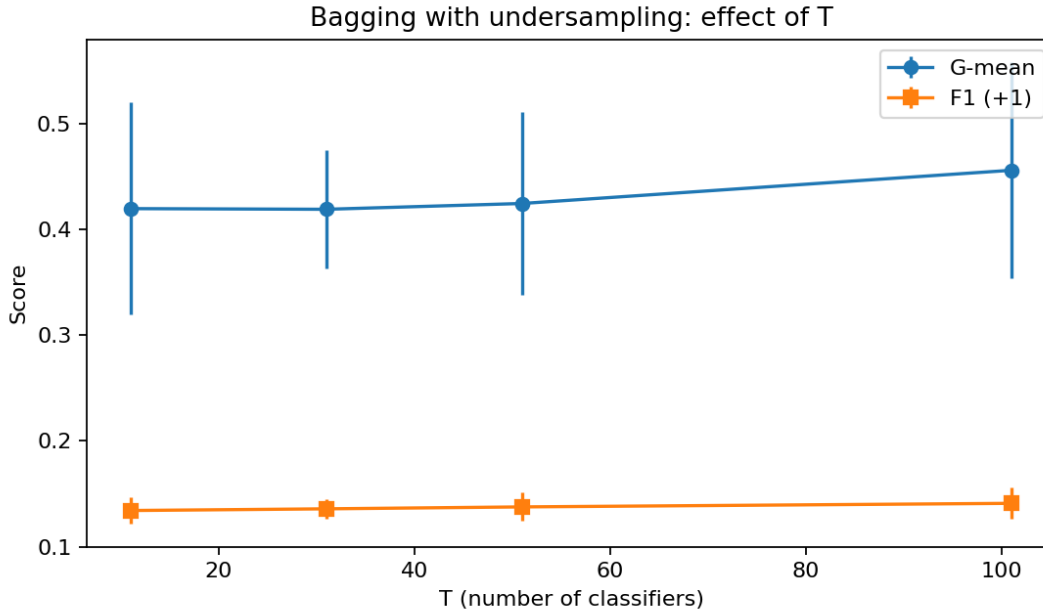


Figure 2. Bagging with undersampling: G-mean and F1 versus ensemble size T .

Increasing T did not monotonically improve all metrics. G-mean was roughly stable from $T=11$ through $T=51$ and rose at $T=101$, while F1 improved only slightly. The improvement after $T=51$ is small relative to the run-to-run standard deviation, so increasing T beyond about 51 gives diminishing returns for F1; by G-mean, $T=101$ is the best among tested values.

Table 6. Bagging base learner comparison at $T=51$

Base learner	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	G-mean	AUC-ROC
HDDT (ours)	0.2477 ± 0.0877	0.0743 ± 0.0081	0.9433 ± 0.0473	0.1376 ± 0.0134	0.4247 ± 0.0868	0.6648 ± 0.0460
Standard DT (sklearn)	0.5277 ± 0.1516	0.0839 ± 0.0159	0.6267 ± 0.1497	0.1467 ± 0.0243	0.5493 ± 0.0736	0.6409 ± 0.0436

The standard CART decision tree as a depth-3 base learner produced higher accuracy, F1, and G-mean than depth-3 HDDT in the undersampled bagging setup. The likely reason is that after undersampling, each training subset is already balanced, so HDDT's class-prior invariance is less decisive, while CART can exploit impurity reductions on the balanced subsets. HDDT in this configuration had very high recall but many false positives, depressing precision and accuracy.

4. Task 3 - AdaBoost.M1 and SMOTE

AdaBoost.M1 was implemented with scikit-learn decision stumps as the weak learners but with custom boosting weights, weighted error, alpha calculation, normalization, and prediction logic.

SMOTE was implemented from scratch using Euclidean k-nearest neighbors among minority samples only and was applied only after each train/test split, never before splitting.

Table 7. AdaBoost versus AdaBoost + SMOTE at T=50

Method	Accuracy	Precision (+1)	Recall (+1)	F1 (+1)	AUC-ROC	G-mean
AdaBoost (no SMOTE)	0.9368 $\pm$ 0.0007	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.0000 $\pm$ 0.0000	0.6113 $\pm$ 0.0267	0.0000 $\pm$ 0.0000
AdaBoost + SMOTE	0.8147 $\pm$ 0.0346	0.0981 $\pm$ 0.0092	0.2367 $\pm$ 0.0711	0.1360 $\pm$ 0.0180	0.5751 $\pm$ 0.0326	0.4427 $\pm$ 0.0619

Without SMOTE, AdaBoost achieved high accuracy but predicted no positive severe cases in these runs, giving zero recall, zero F1, and zero G-mean. Applying SMOTE reduced accuracy but made the classifier sensitive to the minority class, improving recall, F1, and G-mean. This illustrates why accuracy is not the right objective for a 15:1 clinical imbalance.

Table 8. AdaBoost + SMOTE: effect of boosting rounds

T	Train error	Test error	F1 (+1)	AUC-ROC	G-mean
10	0.2640 $\pm$ 0.0262	0.3210 $\pm$ 0.0651	0.1315 $\pm$ 0.0178	0.5530 $\pm$ 0.0177	0.5131 $\pm$ 0.0518
25	0.1949 $\pm$ 0.0331	0.2521 $\pm$ 0.0467	0.1258 $\pm$ 0.0245	0.5770 $\pm$ 0.0310	0.4704 $\pm$ 0.0735
50	0.1357 $\pm$ 0.0196	0.1853 $\pm$ 0.0346	0.1360 $\pm$ 0.0180	0.5751 $\pm$ 0.0326	0.4427 $\pm$ 0.0619
100	0.1083 $\pm$ 0.0176	0.1481 $\pm$ 0.0372	0.1431 $\pm$ 0.0481	0.6076 $\pm$ 0.0479	0.4089 $\pm$ 0.0729

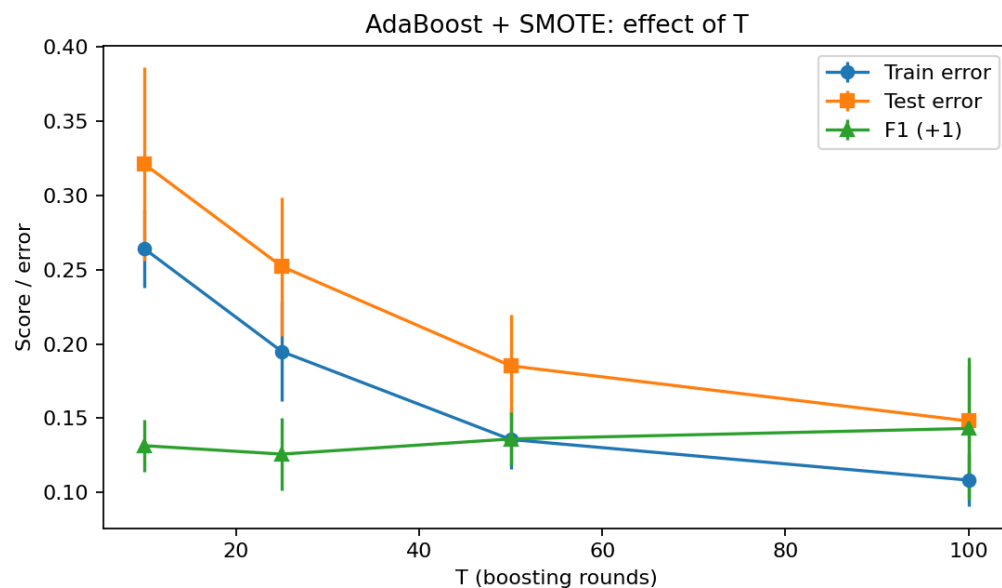


Figure 3. AdaBoost + SMOTE: train error, test error, and F1 versus boosting rounds T.

As T increases, training and test error decrease, but G-mean decreases from T=10 to T=100. This suggests that extra boosting rounds shift the decision boundary toward accuracy on the majority class rather than balanced sensitivity/specificity. F1 is highest at T=50 among the tested values, while G-mean is highest at T=10.

5. Full Comparison Across Methods

Table 9. Best configuration summary by minority F1

Method	Best config	F1 (+1)	G-mean	AUC-ROC
HDDT (Task 1)	max_depth=None	0.1023	0.2710	0.5846
Bagging + HDDT (Task 2)	T=101, max_depth=3	0.1410	0.4561	0.6704
Bagging + DT (Task 2)	T=51, max_depth=3	0.1467	0.5493	0.6409
AdaBoost (Task 3)	T=50, no SMOTE	0.0000	0.0000	0.6113
AdaBoost + SMOTE (Task 3)	T=100	0.1431	0.4089	0.6076

By minority F1, the best method was Bagging + standard DT at T=51, max_depth=3. It had the highest F1, G-mean, and AUC-ROC among the final comparison entries. The result is consistent with the ensemble design: undersampling exposes the learner to severe cases in every balanced subset, while bagging creates diversity through both bootstrap sampling and different majority-class subsets. AdaBoost without SMOTE failed on the minority class; AdaBoost + SMOTE helped but remained weaker than undersampled bagging in F1.

6. Theory Questions

Q1. Hellinger Distance vs. Gini vs. Information Gain

(a) For a node S, the **Gini impurity** is:
$$\text{Gini}(S) = 1 - \sum_c p_c^2$$

A binary split into left child L and right child R is usually scored by the weighted impurity decrease:

$$\Delta \text{Gini} = \text{Gini}(S) - \frac{|L|}{|S|} \text{Gini}(L) - \frac{|R|}{|S|} \text{Gini}(R)$$

The **entropy** of a node is:
$$H(S) = - \sum_c p_c \log_2(p_c)$$

The **information gain** of a binary split is:
$$\text{IG} = H(S) - \frac{|L|}{|S|} H(L) - \frac{|R|}{|S|} H(R)$$

The **Hellinger split score** instead compares the normalized positive and negative class distributions across the left and right children:

$$H_f = \left(\sqrt{\frac{p_L}{P}} - \sqrt{\frac{n_L}{N}} \right)^2 + \left(\sqrt{\frac{p_R}{P}} - \sqrt{\frac{n_R}{N}} \right)^2$$

where:

$$P = p_L + p_R$$

$$N = n_L + n_R$$

Structurally, Gini impurity and information gain are weighted by child size, while Hellinger instead compares the normalized positive and negative distributions across the left and right children using square-root differences. Structurally, Gini and information gain are weighted by child size, while Hellinger uses class-count normalization by P and N.

(b) In imbalanced data, child-size weighting can favor splits that improve purity mostly for the majority class because the majority dominates $|S|$, $|L|$, and $|R|$. Hellinger uses $\frac{p_L}{P}$, $\frac{p_R}{P}$, $\frac{n_L}{N}$, and $\frac{n_R}{N}$, so each class is normalized by its own total at the node. This makes the split score less sensitive to the prior class ratio.

(c) Compared with a standard unbalanced decision tree expectation, HDDT should be less majority-biased. In our Task 1 results, however, shallow HDDT still predicted mostly -1, while the unrestricted HDDT achieved mean recall 0.0833 for +1. It improved minority detection over trivial majority prediction, but the gain was limited by the very small number of positive examples and heavy missingness.

Q2. Stable, Unstable, and Weak Classifiers

(a) A stable classifier changes little when the training data are perturbed; examples include k-nearest neighbors with large k or strongly regularized linear models. An unstable classifier changes substantially under small data changes; a decision tree is the standard example.

(b) Bagging helps most with unstable classifiers because bootstrap samples induce different fitted models. Averaging or voting over them reduces variance while preserving much of the base learner's low bias. If the classifier is already stable, bootstrap samples produce very similar models, so bagging adds little.

(c) A weak classifier is only slightly better than random guessing under the current weighted distribution. AdaBoost relies on weak learners so that each round corrects a different part of the weighted error landscape. A fully grown deep tree may overfit the current weights, reduce diversity between rounds, and produce overly confident corrections that can amplify noise.

Q3. Why Max Depth Must Be Limited in Bagging

- (a) Unlimited-depth base HDDTs can overfit the small balanced subsets created by undersampling. With about 70 minority cases and 70 majority cases per subset, a deep tree can memorize idiosyncratic points rather than learning robust severe-case patterns.
- (b) Diversity means ensemble members make different errors. Bootstrap sampling creates diversity by changing which minority samples appear and their multiplicities. Undersampling adds diversity by giving each learner a different subset of the many majority cases.
- (c) If all base classifiers are identical, majority voting returns exactly that same classifier for every sample. The ensemble error is then equal to the individual classifier error, because no averaging benefit is possible when diversity is zero.

Q4. SMOTE and Data Leakage

- (a) Data leakage occurs when information unavailable at training time or evaluation time is accidentally used to train the model. It causes overly optimistic test results and poor real-world generalization.
- (b) Applying SMOTE before splitting leaks because synthetic minority points are generated from the full dataset. A synthetic point in the training set may be an interpolation involving a future test sample, or a near-duplicate of a test-region minority case. The training process has then indirectly seen test-set structure.
- (c) If SMOTE were applied before splitting, reported AUC-ROC would generally be higher than the true value. The model would see synthetic samples that encode information from test minority observations, making ranking and classification on the test set artificially easy.

Q5. Reflection - Dataset-Specific Challenges

- (a) We used median imputation for age and PO2 and mode imputation for binary/categorical features. This respects feature types and is robust for simple clinical tabular data.
- (b) Missingness may be informative: a symptom not recorded or a clinical measurement missing may correlate with triage, disease severity, or hospital workflow. A simple extension is to add a binary missing-indicator feature for every column with missing values before imputation.
- (c) In real clinical deployment, we would consider calibration, false-negative risk, subgroup fairness, interpretability for clinicians, stability across hospitals, monitoring for distribution shift, privacy, and whether the model improves clinical workflow rather than merely optimizing a benchmark metric.